

Evals: How We Know Whether It Works

FRIDAY AI Education — Episode 008 · Printable Companion Paper

Abstract

A model that produces fluent, confident text is not the same as a system that does its job. The gap between those two things — between "sounds right" and "is right, reliably, on the inputs that matter" — is where most AI projects quietly fail. **Evals** (evaluations) are how you measure that gap and watch it over time. They are the difference between deploying a system because you have evidence and deploying it because the demo went well. This paper explains what evals are, the mechanism underneath them, and the specific instruments you use to build trust: the **golden set**, the **regression test**, **precision and recall**, the **hallucination check**, and **source faithfulness**. It is written for someone building a company on top of these systems, not running a benchmark for a paper. The argument is simple: you cannot govern, iterate on, or stake your reputation on a system you cannot measure. Evals are the measurement. Everything else — the agents, the memory, the automation — sits on top of that foundation or it sits on sand.

1. Why this matters before anything else works

Here is the situation you are actually in when you ship AI inside an organization.

You build something — say, a knowledge base that answers questions about a person's public body of work, your first wedge. You test it by asking it a few questions. It answers well. You show someone. They ask a question. It answers well. Everyone nods. You ship.

Three weeks later, someone asks a question that is slightly off the path you tested, and the system invents a confident, plausible, completely fabricated answer. A customer reads it. Now you have a problem that is not technical — it is a problem of *trust*, and trust is the only currency an operating layer like FRIDAY actually trades in. Once a person catches your system lying once, they re-check everything it tells them forever, which means the system has stopped saving them time, which means it has stopped being worth running.

The uncomfortable truth is that the demo told you almost nothing. A demo measures whether the system *can* produce a good answer on an input you chose. It does not measure whether it *reliably* produces good answers on the inputs you didn't choose — which is the entire job. Language models are fluent by construction. Fluency is not correctness. A wrong answer and a right answer can be equally well-written, equally confident, equally calm. You cannot tell them apart by reading them, and neither can your users — that is the whole danger.

So you need an instrument that measures correctness independently of fluency, across many inputs, repeatably, before and after every change. That instrument is an eval. This is not a nice-to-have you add once the product is mature. It is the thing that lets the product *become* mature, because without it every change you make is a guess and every deployment is a coin flip you can't see the result of until a user does.

There is a second reason, specific to the kind of thing you are building. FRIDAY is meant to be an orchestrator that takes actions — not a chatbot you read and then decide what to do. The moment a system acts on the world (sends the reply, books the meeting, updates the record), the cost of being wrong stops being "the user reads something bad" and becomes "something bad happened." You do not let a system take actions on a real organization unless you can state, with evidence, how often it is right and how it fails. Evals are how you produce that statement. They

are the precondition for earned autonomy.

2. What an eval actually is

Strip away the tooling and an eval is three things:

1. **A set of inputs** you care about getting right.
2. **A definition of what "right" means** for each one.
3. **A repeatable procedure** that runs your system on those inputs and scores how close it got.

That is the whole concept. An eval is a test suite for a system whose outputs are judgments rather than exact values. In ordinary software, a test asserts `add(2, 2) == 4` — the right answer is a single known value and the check is exact equality. In an AI system, the "right answer" to *"What did this person argue about pricing power in 2019?"* is not one string. It is a space of acceptable answers (the ones that are accurate, grounded in the source, and responsive) and a space of unacceptable ones (wrong, fabricated, evasive, unsafe). The eval's job is to decide which space a given output landed in, and to do it the same way every time so the number means something across runs.

This is harder than equality-checking, and the difficulty is the reason people skip it — which is exactly why doing it is an advantage. The hard part is *defining* the answer space. Most of the value of an eval is created before any code runs, in the act of writing down what a good answer looks like. That act forces you to be specific about the job in a way that demos never do. "It should answer questions about his work well" is not a specification you can test. "Given a question about a public claim, the answer must (a) be factually consistent with the source documents, (b) cite the specific source, and (c) say 'I don't have that' rather than guess when the source is silent" — that is testable, and writing it down is itself a clarifying act of product design.

How is the scoring done? Three families, used in combination:

- **Exact or rule-based checks.** Did it cite a real document ID that exists? Did it refuse the out-of-scope question? Did it stay under the length limit? Did it avoid a forbidden action? These are cheap, fast, deterministic, and where you should put as much of your eval as possible, because a check that can't waver is a check you can trust.
- **Reference comparison.** You wrote down an ideal answer or a set of required facts ahead of time, and you measure overlap — does the output contain the key points, does it contradict any of them.
- **Model-graded ("LLM-as-judge").** You ask a separate model to score the output against a rubric: *"Is this answer fully supported by the provided source? Answer yes/no and quote the supporting span."* This scales to fuzzy judgments that rules can't capture, but it has a sharp limitation we return to in Section 9 — a judge is itself a fallible system, so you must validate the judge before you trust the grades, or you have simply moved the trust problem one step away and hidden it.

The point is not to pick one. It is to use rules wherever a rule will do, references where you can write them, and a judge only for the genuinely subjective remainder — and to know which is which.

3. The golden set: the thing you measure against

An eval is only as good as the inputs in it. The **golden set** is the curated collection of test cases — inputs paired with their known-good expected outcomes — that you treat as the standard your system must meet. "Golden" because it is the reference; everything is measured against it.

A golden set is not "a bunch of questions." It is a deliberately constructed sample of the work the system actually faces, and its construction is where the engineering judgment lives. Three properties make it worth trusting:

It covers the real distribution, including the ugly parts. If 30% of real questions to your public-content KB are vague, half-formed, or about things the source never addresses, then roughly 30% of your golden set should be too. A golden set made only of clean, well-posed questions will give you a beautiful score and a system that collapses on contact with real users — because real users are not your QA team and do not phrase things the way you did. The most valuable cases are usually the hard ones: the question the source *almost* answers but doesn't, the adversarial phrasing, the request that should trigger a refusal.

Each case has a defensible expected outcome. For a factual question, the expected answer or the required facts. For an out-of-scope question, the expected refusal. For a borderline action request, the expected boundary — *do this, but not that*. If you cannot say what the right outcome is for a case, you cannot score it, and it does not belong in the golden set until you can.

It grows from real failures. The first golden set you write from imagination is a starting point and is always too optimistic, because you cannot imagine the inputs you've never seen. The valuable golden set is the one that accretes over months: every time the system fails in production, that failing input — with its now-known correct outcome — becomes a permanent case. The golden set becomes an institutional memory of every way the system has ever been wrong, which is precisely the memory you need so it can never be wrong that way again silently.

A practical scale note, because the instinct is to overbuild: you do not need ten thousand cases to start. A focused golden set of 50–150 well-chosen, well-labeled cases that genuinely span your distribution is worth more than a giant pile of repetitive easy ones, and it is small enough that a person can actually read through the failures and understand them. Quality and coverage beat raw count, and a golden set you'll actually maintain beats an ambitious one you abandon.

4. The regression test: catching the change that breaks the old thing

Once you have a golden set, the single most valuable thing it buys you is the ability to detect **regressions** — cases that used to work and silently stopped working after a change.

This failure mode is specific to AI systems and it is vicious. In ordinary software, a change that breaks something usually breaks it loudly: an error, a crash, a failed assertion. In an AI system, a change — a new prompt, a swapped model, an updated retrieval step, a tweaked instruction — can improve ten things and quietly break three others, and *nothing crashes*. The system keeps producing fluent, confident output. The output is just now wrong on inputs it used to handle correctly, and you will not find out by looking, because wrong output looks exactly like right output. You will find out when a user does, which is the worst possible time and the most expensive possible way.

A **regression test** is the defense: you run the full golden set before a change and after it, and you compare case-by-case. Improvements are expected. What you are hunting for is the case that went from pass to fail — the silent breakage. The discipline is non-negotiable: *no change to a deployed AI system ships without running the golden set and comparing against the previous run*. This is the AI equivalent of "don't merge with failing tests," and it is the practice that most separates teams that improve steadily from teams that thrash — that fix one thing on Monday, break two on Wednesday, and never quite know whether the system is getting better or worse because they have no instrument that measures direction.

There is a deeper organizational point here, and it is the one most worth internalizing. The reason teams iterate confidently is not that they are smart. It is that they have a number that goes up, and a mechanism that screams when it goes down. Without that, every change is a debate about vibes, and vibes do not compound. With it,

improvement becomes a ratchet — each gain is locked in, protected by a regression test, and built upon. That ratchet is what lets a system get genuinely good over time instead of oscillating forever around "seems okay." Evals are how you turn iteration from a hope into a process.

5. Precision and recall: the two ways to be wrong

You cannot reason about correctness with a single number, because there are two fundamentally different ways to be wrong and they trade off against each other. **Precision** and **recall** are the language for this, borrowed from information retrieval, and they are worth holding precisely because the trade-off between them is a *business decision disguised as a metric*.

Take the concrete case of your public-content knowledge base retrieving source passages to answer a question.

- **Precision** asks: *of the things the system returned, how many were actually correct/relevant?* High precision means few false positives — when it gives you something, you can trust it. Low precision means it returns junk mixed with gold and you have to sort it yourself, which defeats the purpose.
- **Recall** asks: *of the things it should have returned, how many did it actually find?* High recall means few false negatives — it doesn't miss the relevant passage that was sitting right there. Low recall means it leaves correct, available information on the table and answers as if it didn't exist.

The reason you need both is that each is trivially gamed by sacrificing the other, and a single number hides the cheat. A system that answers only when it is absolutely certain has near-perfect precision and terrible recall — it is *right* but useless, because it refuses most of the time. A system that returns everything plausibly related has high recall and low precision — it never misses but buries the answer in noise, so you can't trust any single thing it says. The job is to hold both at acceptable levels, and "acceptable" is defined by the use case, not by the math:

Use case	Cost of low precision (false positive)	Cost of low recall (false negative)	Where to bias
Public KB answer	Confidently states something wrong → erodes trust	Misses a relevant passage → answer is thinner	Precision — a trusted-but-incomplete KB beats a complete-but-untrusted one
Support reply safety	Sends an unsafe/incorrect reply → real-world harm	Escalates a question it could've handled → human does more	Precision , hard — false positives reach a customer
Compliance / risk flagging	Flags a clean item → wasted human review	Misses a real violation → the thing you were guarding against happens	Recall — a missed violation is the whole failure
Meeting summary recall	Includes a point that wasn't decided → misinformation	Omits a real decision → it's lost	Balanced, leaning recall for decisions/actions

Read the table as a decision tool, not a reference. The discipline it enforces is this: for every workflow, *before* you build, decide which error is more expensive and bias the system — and the eval that grades it — toward avoiding that one. A system tuned without that decision is tuned toward whatever the model happened to do, which is to say tuned toward nothing.

6. The hallucination check: confident invention

A **hallucination** is output that is fluent, confident, and not true — facts the source never contained, citations to documents that don't exist, claims with no basis. It is the defining failure mode of language models and the reason naïve deployments are dangerous, because the model has no built-in signal that distinguishes "I am recalling something grounded" from "I am generating something plausible." Both feel identical from the inside, and both read identical from the outside.

A **hallucination check** is an eval specifically targeting this. The mechanism rests on one structural insight: a hallucination is, almost always, a claim that is *not grounded in any provided source*. That makes it detectable in a way that "is this true in general" is not — you do not need to know all of human truth, you only need to check whether each claim in the output is supported by the documents the system was actually given. The check becomes: **decompose the output into its claims, and for each claim, verify that a provided source supports it**. Any claim with no supporting source is a hallucination candidate, regardless of whether it happens to be true in the world — because a claim the model couldn't justify from its sources is a claim you got lucky on, and luck does not survive scale.

You implement this with a mix of methods from Section 2. Citation existence is a cheap rule: did every cited document ID resolve to a real document? Span-grounding is a judge task: *"Does this provided passage actually support this claim? Quote the words that do."* The forcing function on the system side — distinct from the eval — is to design it so it can only speak from retrieved sources and must say "the source doesn't address this" when they're silent. This is the entire reason retrieval-augmented generation (**RAG** — retrieval augmented generation) exists: you constrain the model to answer *from* fetched documents rather than from its own parameters, which both improves grounding and, crucially, *makes hallucination checkable*, because now there is a defined source set to check claims against. The eval and the architecture work as a pair — the architecture creates the ground truth, the eval enforces that the output stays on it.

For your wedge specifically, this is the whole game. A public-content knowledge base that occasionally fabricates a quote or invents a position the person never held is worse than no product, because it launders fiction through the authority of a real person's name. The hallucination check is the eval that stands between "searchable, cited, trustworthy body of work" and "confident misinformation generator wearing someone's reputation."

7. Source faithfulness: did it report, or did it editorialize?

Source faithfulness is the close cousin of the hallucination check, and the distinction is worth holding precisely because the failures are different and both are real.

A hallucination invents something that *isn't in the source*. A faithfulness failure *distorts something that is*. The system was given the right material and still misrepresented it — it overstated a hedged claim, dropped the qualifier that changed the meaning, merged two separate points into one that neither original supports, or "summarized" a tentative argument into a firm conclusion. No fact was invented. The meaning was bent. And a bent meaning is harder to catch than an invented fact, because every individual word can be traced to the source — it's the *shape* that's wrong.

A faithfulness eval checks whether the output is a true representation of what the source *actually* says, including its hedges, its scope, and its uncertainty. The questions are sharper than "is this supported": Did a "might suggest" become a "proves"? Did "in this specific case" become "in general"? Did a summary preserve the parts that complicate the headline, or did it quietly launder nuance into a cleaner, falser story? The model-graded version compares output against source span-by-span and asks not "is this in the source" but "does this *fairly represent* the source," which is a genuinely higher bar.

This matters most exactly where you most want to use these systems. A meeting summary is a faithfulness artifact: its entire value is being a true compression of what happened, and a summary that says a decision was made when

it was merely discussed is faithful-looking and wrong in a way that propagates — people act on the summary, not the meeting. A public-content KB is a faithfulness artifact: representing someone's actual positions, hedges intact, is the product; flattening them is the failure. Where summarization, representation, and compression are the job, faithfulness is the metric that defines whether you did the job or only appeared to.

8. How this becomes an organizational capability

Now connect the instruments to the thing you're building, briefly, because the mechanism is the lesson and FRIDAY is only the application.

FRIDAY is meant to orchestrate and act, on top of GBrain as the knowledge and memory substrate, with the model as a capable but bounded reasoning engine. Useful organizational AI is never just the model — it is the model plus context, memory, tools, permissions, *evals*, audit, and human approval gates. Evals are load-bearing in that list for one reason: every other governance mechanism depends on knowing how well the thing works. You cannot set a sensible approval gate without knowing the system's error rate. You cannot grant a tool permission responsibly without knowing how often the system uses it correctly. You cannot audit what you never measured. Evals are the measurement layer the whole governance stack reads from.

This reframes the most important question in deploying agents — **agent action boundaries**, the line between what the system may do alone and what needs a human. That line should not be set by intuition. It should be set by evidence, and the evidence is an eval. *On the action "send a customer support reply," the system scores 0.99 precision on safety across the golden set, so it may send routine replies but must escalate anything it flags as uncertain.* That is a defensible boundary because a number backs it, the number came from a representative golden set, and a regression test guards it so it cannot silently rot. Move the boundary outward — more autonomy — only when the eval earns it. Pull it back the moment a regression says the system slipped. The boundary becomes a function of measured competence rather than hope, which is the only basis on which a serious person will let your system touch their organization.

GBrain closes the loop. The golden set, the production failures, the regression history — these are organizational memory about how the system behaves, and they belong in the memory substrate, not in a forgotten file. A system that remembers every way it has been wrong, and tests against that memory before every change, is a system that compounds. That is the difference between AI that gets more trustworthy each quarter and AI that has to re-earn trust after every change forever — and the difference is entirely whether you built the evals.

9. The main limitation: your eval can be wrong, too

The honest section. An eval is a measurement instrument, and like any instrument it can be miscalibrated. Trusting a bad eval is worse than having none, because it manufactures false confidence — it gives you a number that goes up while the system gets worse, and a number you trust wrongly is more dangerous than no number at all. The failure modes, plainly:

The golden set doesn't match reality. If your cases don't represent real inputs — too clean, too narrow, drawn from your imagination instead of your logs — your score measures performance on a world that doesn't exist. You score 0.95, ship, and discover the real distribution was nothing like your test set. This is the most common and most seductive failure, because the high score feels like progress. The defense is discipline about coverage and a golden set fed continuously by real failures, never frozen the day you wrote it.

The judge is fallible. Model-graded evals move the trust problem rather than solving it: now you trust the judge. Judges have biases — they favor longer answers, get fooled by confident tone, miss subtle faithfulness violations,

the very things you most need them to catch. The defense is to validate the judge against human labels on a sample before trusting it, prefer cheap deterministic rules wherever they suffice, and never let a model grade the high-stakes cases without a human spot-checking the grader. An unvalidated judge is an opinion with a number attached.

Evals drift out of date. The world changes, the product changes, the inputs change, and a golden set written six months ago can quietly stop representing the job — passing on questions no one asks anymore while real traffic moves on. Evals are living artifacts. A stale eval gives a confident score about a problem you no longer have.

Goodhart's law. When a measure becomes a target, it stops being a good measure. Optimize hard enough against a fixed eval and you get a system tuned to that specific set rather than to the underlying job — it learns the test, not the task. The defense is to keep a portion of your cases held out and rotate in fresh ones, so you are always measuring generalization rather than memorization.

None of this argues against evals. It argues that evals are a discipline, not a one-time artifact — something you maintain, validate, and treat with the same skepticism you apply to the system they measure. The person who says "we have evals" and stops thinking is more dangerous than the person who has none and knows it. Measure, then doubt the measurement, then improve it. That loop is the actual competence.

10. Vocabulary

- **Eval (evaluation)** — A repeatable procedure that runs a system on a set of inputs and scores how well its outputs meet a defined standard. The measurement instrument for systems whose outputs are judgments.
- **Golden set** — A curated collection of inputs paired with known-good expected outcomes, representative of real work, used as the standard a system is measured against. Grows from real failures.
- **Regression / regression test** — A case that used to pass and silently started failing after a change; the test that catches it by comparing golden-set results before and after every change.
- **Precision** — Of the things the system returned, the fraction that were actually correct/relevant. Low precision = confident junk (false positives).
- **Recall** — Of the things it should have returned, the fraction it actually found. Low recall = misses things that were right there (false negatives).
- **Hallucination** — Fluent, confident output not grounded in any provided source. **Hallucination check** — an eval that decomposes output into claims and verifies each is source-supported.
- **Source faithfulness** — Whether output truthfully represents what the source actually says — including hedges, scope, and uncertainty — without distortion. Distinct from hallucination: faithfulness bends real material; hallucination invents new.
- **LLM-as-judge** — Using a separate model to grade outputs against a rubric. Scales to subjective judgments; must itself be validated against human labels before trusted.
- **RAG (retrieval augmented generation)** — Constraining a model to answer from retrieved documents rather than its own parameters. Improves grounding and makes hallucination checkable by defining a source set.
- **Agent action boundary** — The evidence-set line between what a system may do autonomously and what requires human approval. Should be a function of measured competence, not intuition.
- **Goodhart's law** — When a measure becomes a target, it ceases to be a good measure. The reason to hold out and rotate eval cases.

11. Reading guide

Two anchors. Read them with a specific question in mind rather than cover to cover.

OpenAI Evals — <https://github.com/openai/evals>

A working framework and registry of real evals, not a paper. Don't try to absorb it all. Browse a handful of existing eval definitions to internalize one thing: an eval is *concrete* — a set of inputs, a way to declare the expected outcome, and a grading method. Notice the split between exact-match style checks and model-graded ones, and notice how much structure is required to make "is this answer good?" into something a machine runs the same way twice. The lesson is the *shape* of an eval, made tangible. Skip the parts about their specific infrastructure; you want the pattern, not the plumbing.

Anthropic — Building effective agents —

<https://www.anthropic.com/research/building-effective-agents>

Read this for where evals sit in the larger picture. Its core argument — favor simple, composable, *measurable* patterns over elaborate autonomy — is the architectural twin of this paper's thesis. Pay attention to how it treats evaluation and guardrails not as an afterthought but as a precondition for letting agents act. Read it asking: *for the action boundaries I want FRIDAY to eventually cross, what would I need to measure first?* The two readings meet there — Evals gives you the instrument, this gives you the judgment about when an instrument has earned you the right to let go.

12. Walking questions

1. Which of these instruments — golden set, regression test, precision/recall, hallucination check, source faithfulness — could you explain cleanly to a sharp non-technical person right now, and which one still goes blurry when you try?
2. For your public-content KB wedge, which error is more expensive: confidently stating something wrong (low precision), or missing a relevant passage that was there (low recall)? What does that answer tell you about how to tune *and* how to grade it?
3. Where in FRIDAY or GBrain does the golden set actually live — and who or what is responsible for feeding it the next real failure so it doesn't go stale?
4. Name one bounded workflow you could put behind an eval-defined action boundary first — narrow enough to label good outcomes honestly, real enough that getting it right matters.

One-sentence takeaway

You cannot trust, iterate on, or safely deploy a system you cannot measure — and evals are the measurement, which makes them not a feature of serious AI work but its precondition.